

MODELLING AND SIMULATION

The logo of the University of Twente, featuring the text "UNIVERSITY OF TWENTE." in white, bold, sans-serif capital letters on a black rectangular background.

UNIVERSITY
OF TWENTE.

FINAL ASSIGNMENT

INTEGRATION: ROBOT ARM WITH ACTUATION AND CONTROL

Group 6

Group Members:

Erik Leering (s2089831)

Rainer Lourens (s3067556)

Harm van Mierlo (s2829126)

Jorrit Kolsteeg (s2613778)

Contents

1	Introduction	3
1.1	Realistic modelling assumptions	4
1.2	Actuator Requirements	4
1.3	Robust and rigorous design and verification	4
2	Model	6
2.1	Theoretical background	6
2.1.1	Model parameters	6
2.1.2	Motor-gearbox selection	7
2.1.3	System inertias	8
2.1.4	Impedance Control	10
2.1.5	Cartesian Impedance Control	11
2.1.6	Series Elastic Actuators (SEAs)	11
2.1.7	Torque Control	13
2.1.8	Gravity compensation	13
2.2	Model overview	13
2.2.1	Modelling choices	14
3	Simulation	19
3.1	Base model simulations	19
3.2	Perfect Impedance Control with gravity compensation	20
3.3	Gravity compensation verification	20
3.4	Perfect SEAs for Impedance Control	21
3.5	Isolated motor simulations	22
3.5.1	Gearbox frictions	22
3.5.2	Current control	23
3.6	Ordered realistic motor-gearbox tuning	25
3.6.1	Continuous Impedance Control	27
3.7	Discrete-time realistic motor-gearbox SEAs for Impedance Control	28
3.7.1	Discrete Current Control	28
3.7.2	Discrete Torque Control	29
3.7.3	Discrete Impedance Control	29
4	Discussion & Conclusion	31
4.1	Discussion of modelling challenges	31
4.1.1	Trajectory tracking and integration error	32
4.1.2	Actuator tuning and friction estimation	32
4.1.3	Power limit motors exceeded	32
4.1.4	Feedforward friction compensation	33
4.2	Conclusion	33

1 Introduction

For the final Modelling & Simulation assignment, we modelled a robotic arm. The robotic arm consists of three bodies with three rotational joints. In terms of kinematics, Joint 1 controls the yaw, while Joints 2 and 3 control the pitch. From bottom to top in fig. 1, we have:

- Body 1** (black): a base on which it stands.
- Joint 1** (red): a rotational joint connecting the first two bodies, which allows rotation around the z-axis.
- Body 2** (yellow): an in-between body, to have two links at the base.
- Joint 2** (red): a rotational joint in the same place as **Joint 1**, allowing rotation around the y-axis.
- Link 1** (green): a first arm.
- Joint 3** (red): a rotational joint connecting the second and third bodies, allowing rotation around the y-axis.
- Link 2** (blue): a second arm.

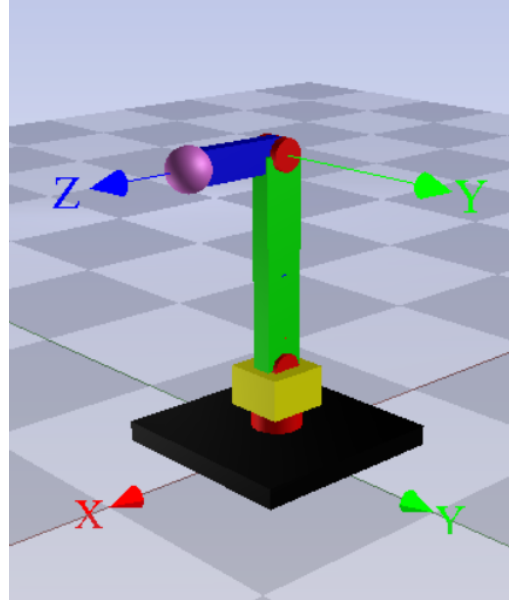


Figure 1: The robotic arm. Filename: *3d_arm_no_alpha.emx*.

Project goals

We decided specifically to model an industrial robotic arm. That is, an arm that is long and heavy, making it appropriate for interactions in real-world industries. We will develop this model and simulation by working towards the following goals:

1. We will make a model and simulation that is realistic:
 - (a) By picking physically appropriate dimensions of the system and values for the relevant parameters, and
 - (b) By incorporating existing motors and gearboxes that are appropriate for our industrial use case.
2. We will design our robotic arm to be appropriate for use cases in human-industrial settings by using Series Elastic Actuators for Impedance Control with appropriate behaviour.
3. We construct our model and the control of our robotic arm with sufficient robustness and rigour:
 - (a) By validating our base robotic arm model before considering any actuation,
 - (b) By tuning the control parameters for each element ordered by dependencies,
 - (c) By validating the performance of each actuator after integration into the system ensemble.

We will elaborate on these goals and explain how we plan to tackle them.

1.1 Realistic modelling assumptions

To perform industrial tasks, the robotic arm should be large enough, while at the same time not becoming excessively heavy. For this purpose, we decided to model Link 1 and Link 2 as hollow cuboids. Furthermore, we decided to assume that the bodies are made out of aluminium.

We will model existing motors and gearboxes from the Maxon catalogue, which is available online [1]. Specifically, we will calculate the maximum torque that the motor-gearbox combination should be able to continuously hold in the extreme configurations of our setup, and choose combinations of motors and gearboxes that achieve this as their nominal holding torque.

1.2 Actuator Requirements

For the robotic arm to be safe in human-industrial environments, we require that the arm gives way when disturbed. To illustrate this, suppose that a person hits this robotic arm with their head. It would be necessary for the robotic arm to mimic the behaviour of a spring-damper system to avoid injuries. To mimic the behaviour of a virtual spring-damper is precisely the goal of Series Elastic Actuators (SEAs) and Impedance Control, which is why we apply this form of control on our robotic arm.

We consider the maximum force a robotic arm can exert on the upper arms and elbows of a human as described in ISO/TS 15066 Annex A safety standards [2], which is 150 N. Our robotic arm will have an operation workspace of up to about that height, and we assume interactions with humans will be with their arms. For safety purposes, we will require our robotic arm to act as a spring, and we will want to reach this force at a stretched state of 50cm.

We are considering the operation of the robotic arm in Cartesian coordinates, so that the virtual spring of our robotic arm should be

$$C = \frac{F_{max}}{\Delta x} = \frac{150[N]}{0.5[m]} = 300[N/m]. \quad (1)$$

1.3 Robust and rigorous design and verification

Although the system ensemble is complicated and the joints are dependent on each other, there is a difference in the measure of the dependence. The dependence of each joint on one another can be illustrated graphically:

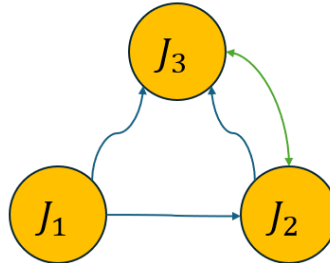


Figure 2: Joint dependencies

Here, a blue arrow indicates positional dependence, whereas a green arrow indicates dependence on velocity. From this, we can sort the joints on a measure of relevance and conclude that we should first tune Joint 3, then Joint 2 and then Joint 1 in our system ensemble.

Furthermore, each of the joints will be actuated by some motor that requires tuning of its current control. The performance of the joints is obviously dependent first on the performance of the current control, which will be tuned first.

2 Model

In the discussion below, we refer to the robotic arm in the configuration shown in fig. 3. We model our robotic arm using the 3D Mechanics Editor of 20-SIM. This model can be found in *3d_arm_no_alpha.emx*.

We make the following modelling assumptions:

1. Body 1 is the fixed world. Its inertia is not taken into account, and it will be considered immovable.
2. We equip fig. 3 with the following reference frames:
 - (a) Ψ_0 , the fixed world reference frame, to the centre of Body 1. (Bottom frame)
 - (b) Ψ_1 to the base of Link 1. It will follow the rotations of Joint 1 and 2. (Middle frame)
 - (c) Ψ_2 to the base of Link 2. Its position is determined relative to that of Ψ_1 , and it follows the rotation of Joint 3. (Top frame)
3. The connections are as follows:

Joint	Rotation	Connection 1	Connection 2
Joint 1	z-axis of Ψ_0	Body 1 at 0.2 m	Body 2 at 0 m
Joint 2	y-axis of Ψ_1	Body 2 at 0.2 m	Link 1 at 0 m
Joint 3	y-axis of Ψ_2	Link 1 at 1 m	Link 2 at 0 m

Table 1: Body-joint definitions and connections.

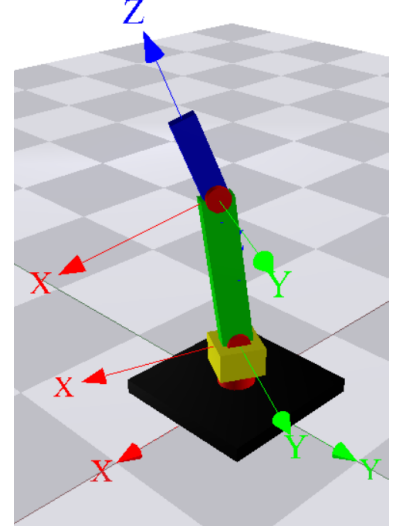


Figure 3: Robotic arm model

Furthermore, Joint 2 is modelled to have a minimum and maximum rotation of ± 90 deg. These extreme angles are enforced by a spring with spring constant 10,000 [N.m] and a damper with Damping constant 100 [N.m.s], which are activated whenever the system starts to exceed its bounds.

Similarly, Joint 3 is set to have bounds on its angle of ± 135 deg, enforced with the same spring-damping system.

2.1 Theoretical background

2.1.1 Model parameters

For industrial use, we decided to try to achieve a large workspace. To this end, we use the following dimensions in our model:

Body	Height L [cm]	Length w_1 [cm]	Width w_2 [cm]	Thickness d [cm]
Link 1	100	20	10	1
Link 2	50	15	7	1

In order to properly model the robotic arm, we need to find the inertias of the links. The moments of inertia of a cuboid around its centre of mass (CM) are given as follows [3]:

$$I_{xx}^{CM} = \frac{m}{12}(y^2 + z^2), \quad I_{yy}^{CM} = \frac{m}{12}(x^2 + z^2), \quad I_{zz}^{CM} = \frac{m}{12}(x^2 + y^2), \quad (2)$$

where the orientation of x, y, z depends on the choice of coordinates. In our case, x is along the length w_1 , y is along the width w_2 and z is along the height L . In these equations, we can find the total mass for a hollow cuboid with non-zero cross-section A as follows:

$$m = LA\rho = L(2w_1d + 2w_2d - 4d^2)\rho, \quad (3)$$

with L, w_1, w_2, d from before and where ρ is the density of aluminium, which is $\rho = 2710 \text{ kg.m}^{-3}$. Eq. (3) can be seen as counting the thickness of each side and then making sure that we are not counting the corners twice each. Inserting eq. (3) into eq. (2) and filling in the values, we obtain:

body	Mass [kg]	I_{xx}^{CM} [kg.m ²]	I_{yy}^{CM} [kg.m ²]	I_{zz}^{CM} [kg.m ²]
Link 1	15.176	1.277	1.315	0.063
Link 2	5.420	0.115	0.123	0.012

Table 2: Link inertia at their CM

These are the inertias that are used in the 3D model *3d_arm_no_alpha.emx*. Note that the inertias experienced by the actuators are not the above. Before we can derive the precise inertias, we will have to select suitable motor-gearbox combinations for our joints and include their weight.

2.1.2 Motor-gearbox selection

The criteria for selecting suitable motors to actuate joints 2 and 3 are the same: we consider the maximum holding torque that the motor and gearbox need to provide, and take this as a minimum for the nominal operating torque. In this case, the maximum holding torque is when the links are stretched out horizontally, as in that case, gravity acts perpendicular to the centre of mass of the body.

The links experience a torque due to gravity:

$$\tau_g = g * m * C_m \sin(\phi), \quad (4)$$

where $g = 9.81 \text{ [m.s}^{-2}\text{]}$ is the gravitational constant, m is the mass of the body which we consider, C_m is the position of the CM of the body with respect to the joint around which we consider the rotation, and ϕ is the angle between the CM and the z-axis of the world frame.

In case of Joint 3, the maximum rest torque it experiences is only due to Link 2, and amounts to 13.293 [N.m]. We decide to actuate this joint using the **EC-4pole 32** motor and the **GPX 70 A** gearbox, the stats of which can be found in table 4-5.

Similarly, Joint 2 experiences a torque by Link 1, but also by Link 2, and by Joint 3. The torque experienced due to Joint 3 depends on the weight of the motor-gearbox combination described above. The sum of all rest torques experienced by Joint 2 amounts to 125.652 [N.m]. In order to actuate this nominal torque, we use the **IDX 70 M** motor and the **GBX 65 WORM** gearhead.

An overview of the rest torques experienced by Joints 2 and 3 is as follows:

Joint	Body	C_m [m]	Mass [kg]	$\tau_g = m.g.C_m \sin(\phi)$ [N.m]	$\tau_{\text{tot,max}}$ [N.m]
Joint 3	Link 2	$\frac{1}{2}L_2 = 0.25$	5.420	$13.293 \sin(\theta_2)$	13.293
Joint 2	Link 1	$\frac{1}{2}L_1 = 0.5$	15.176	$37.219 \sin(\theta_1)$	125.652
	Joint 3	$L_1 = 1$	2.240	$21.970 \sin(\theta_1)$	
	Link 2	$L_1 + \frac{1}{2}L_2 = 1.25$	5.420	$66.463 \sin(\theta_1 + \theta_2)$	

Table 3: Joint 2 and 3 maximum rest torques.

Motor	Weight	K_t	Max. volt / Nom. torque	R	I
EC-4pole 32	0.860 kg	0.0705 N.m/A	48 V, 480 Watt 0.804 N.m	1.01 Ω 0.034 N.m	0.298 mH 140 g.cm ²
IDX 70 M	2.295 kg	0.124 N.m/A	48 V, 890 W 2.530 N.m	0.0715 Ω 0.101 N.m	0.246 mH 1050 g.cm ²

Table 4: Motor stats

Note that the parameter values for the EC-4pole 32 motor vary for each possible stage. This is not reflected in the data sheets that can be downloaded from the Maxon website. Whenever the datasheet and the information for the individual gearbox conflict, we have opted to use the information for the specific stage as listed on the Maxon catalogue online.

Gearhead	Nom. torque	Ratio	Efficiency	Friction	Weight	Inertia
GPX 70 A	30 N.m	21:1	92%	0.0226 N.m	1.380 Kg	128.8 g.cm ²
GB 65 Worm	120N.m	80:1	70%	0.0267 N.m	3.500 kg	< 1 g.cm ²

Table 5: Gearbox stats.

It should be noted that the friction inside the gearbox is not given. This was found using the simulations of section 3.5.1.

The inertia of the GB 65 Worm gearbox scales with the inverse of the reduction ratio squared, and so it becomes negligible with respect to the inertia of the motor.

Lastly, joint 1 does not need to hold any heavy weights against gravity; it only needs to rotate the links. We determined the torque requirements for the motor and gearbox of joint 1 by using the following formula: $\tau = I \cdot \alpha$. By deciding a maximum angular velocity the arm should rotate at, and taking the inertia joint 1 has to actuate in the fully extended configuration (7). It was decided we want the arm to move at around $\frac{\pi}{2} \text{ rad/s}$, which resulted in a minimum nominal torque for the motor and gearbox of: $\tau = 14.578 \cdot \frac{\pi}{2} \approx 22.9 \text{ N.m}$. Since this requirement is similar to joint 3 it was decided that it suffices to use the same motor-gearbox combination as we are using for joint 3.

Note : Deviation from physical implementation

In our setting, we are relying fully on our actuators to keep our robotic arm in place under every configuration. In realistic implementations, brakes are used in a similar way to how we implement our limit angles: if the task does not require rotation in the large arm, then it can be locked in place, saving a lot of work for the motor.

Next to this, real implementations may use other tools, such as counterweights. This is not taken into consideration for our model.

2.1.3 System inertias

As was discussed before, it is best to first tune the actuation of Joint 3, then Joint 2, then Joint 1. In order to tune the control of these joints, we will first model them separately, following only some reference signal and working on some load. These loads will be the maximum inertias experienced within the operational range of the robotic arm.

One should note that the links in our model are actuated at their base and not at their centres of mass. That is, the inertias experienced by the SEAs are not those of table 2. To correct for this, we apply the parallel axis theorem, which states that the moment of inertia at the point of actuation around some axis is given by

$$I_{jj}^{base} = I_{jj}^{CM} + m\delta_{jj}^2, \quad (5)$$

where δ_{jj} is the distance between j -axis through the point of actuation and the same (hence parallel) j -axis through the CM. In our case, we can define the origin to be in the point of actuation. Then, the CM will have coordinates $(0, 0, \frac{1}{2}L)$. The distance between the point of actuation and the CM is only measured in their height, so this distance lies along the z -axis. That is, both $\delta_{xx} = \delta_{yy} = \frac{1}{2}L$. The z -axis goes through the origin and the CM, and so $\delta_{zz} = 0$, leaving this moment of inertia unchanged.

Lastly, we model the joints as point masses, so that their inertia is always given by $I_{jj} = m\delta_{jj}^2$, with once again δ_{jj} the distance to the axis of rotation. We take the base of Link 1 as the actuation point for Joint 3. We obtain the following inertias around the bases of the links:

body	Mass [kg]	I_{zz} [kg.m ²]	I_{xx} [kg.m ²]	I_{yy} [kg.m ²]
Link 1	15.176	0.063	5.071	5.109
Link 2	5.420	0.012	0.454	0.462
Joint 3	2.240	0	2.240	2.240

Table 6: Link moments of inertia around their base

Joint 3 is at the base of Link 2 and only actuates Link 2, and so it always experiences the inertia I_{yy} of Link 2 as in table 6.

The load that Joint 2 experiences is the I_{yy} inertia due to Link 1 and Link 2, as well as the I_{yy} inertia due to the weight of Joint 3. These inertias can be added, and are given as follows:

1. The inertia due to Link 1 is the I_{yy} as in table 6.
2. The inertia due to Joint 3 is the I_{yy} as in table 6.
3. The inertia due to Link 2 is configuration-dependent. The distance δ_{yy} of the parallel line through the CM of Link 2 with respect to the rotational axis through the joint is just the distance from the joint to the CM of Link 2. Denoting the base of Joint 2 as $(0, 0, 0)$, the tip of Link 2 as $(0, 0, L_1)$, we find the position of the CM to be just $(\frac{1}{2}L_2 \sin(\theta), 0, L_1 \sin(\theta))$, to find the distance $\delta_{jj}^2 = L_1^2 + \frac{1}{4}L_2^2 + L_1L_2 \cos(\theta)$. We will use tune the controllers with respect to the maximum load, which here would be at $\cos(\theta) = 1$ to give $I_{yy} = 8.592$ [kg.m²].

The max load that Joint 2 experiences is thus an inertia of $5.109 + 2.240 + 8.592 = 15.941$ [kg.m²].

Similarly, the load that Joint 1 experiences is configuration dependent, but now it varies over both angles θ_1 of Link 1 w.r.t. the rotational z -axis of Joint 1, and θ_2 , which is the angle between Link 1 and Link 2. We may, for simplicity, align our x - and y - axes such that Link 1 and 2 lie in the xz -plane and rotate the x - and y -axes as we rotate around the z -axis. In that case, δ_{yy}^2 is just the x -coordinate of our centres of mass. This is given as follows:

1. The CM of Link 1 is at $x_1 = \frac{1}{2}L_1 \sin(\theta_1)$. The maximum inertia is $I_{zz,1} = 0.063 + 15.176 * (0.5)^2 = 3.857$ [kg.m²].
2. The CM of Joint 3 is at $x_2 = L_1 \sin(\theta_1)$. The maximum inertia is $I_{zz,2} = 2.240$ [kg.m²].

3. the CM of Link 2 is at $x_3 = x_2 + \frac{1}{2}L_2 \sin(\theta_1 + \theta_2)$. The maximum inertia is $I_{zz} = 0.012 + 5.420 * (1.25)^2 = 8.481 \text{ [kg.m}^2\text{]}$.

Now the maximum load is in the configuration where each component is aligned horizontally, so that $\theta_1 = \theta_1 + \theta_2 = \pi/2$, for which we find an inertia $I_{zz} = I_{zz,1} + I_{zz,2} + I_{zz,3} = 14.578 \text{ [kg.m}^2\text{]}$.

To summarise, we wil tune the control of our SEAs using the following loads:

Joints:	Joint 1	Joint 2	Joint 3
Load:	14.578 [kg.m ²]	15.941 [kg.m ²]	0.462 [kg.m ²]

Table 7: Experienced loads for SEA-controller tuning

2.1.4 Impedance Control

Impedance Control is a form of control where it is not the goal to perfectly track a certain trajectory. Instead, its goal is to mimic the impedance of a specified element, in our case, a spring-damper. We briefly describe its simplest implementation.

Consider the desired virtual spring stiffness derived in eq. (1). To this, we add a virtual damper of $170 \text{ [N.s.m}^{-1}\text{]}$. This damping value is slightly above the critically damped value which was derived with the formula: $B_c = 2\sqrt{km} = 2\sqrt{300 \cdot (15.176 + 5.42 + 2.24)} = 165.5 \text{ N.s.m}^{-1}$, where k is the virtual spring stiffness and m the mass of our robot. To ensure a safety margin and keep the calculation more simple, we used the total mass of the arm rather than dynamically calculating the perceived mass at the end-effector based on its current configuration. The reason that we want a slightly overdamped system is that the higher damping coefficient ensures a safety margin compared to a critically damped system that prevents any overshoot or bouncing when the robot collides. If a larger safety margin is desired, the damping coefficient in the impedance controller could be increased.

A physical instantiation of such a spring-damper system acting on a certain load would be the following:

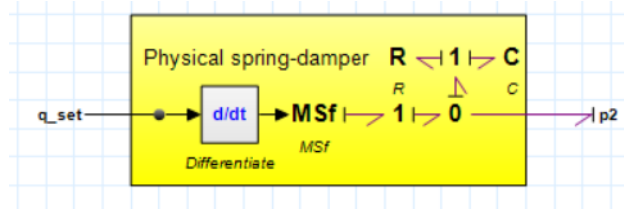


Figure 4: *G06_impedance-overview.emx* $\rightarrow$ *physical*. Physical spring-damper implementation

In reality, there are no physical springs and dampers with these parameters included in the system. Instead, we will incorporate Impedance Control to mimic this virtual spring and damper. A controller that would perfectly mimic this behaviour would be the following:

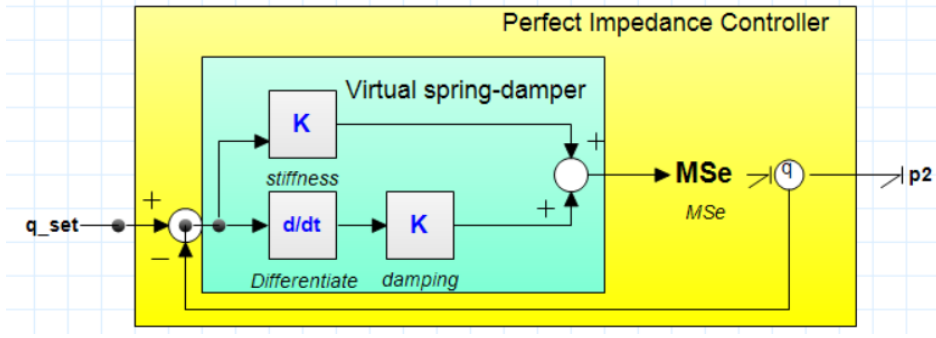


Figure 5: *G06_impedance-overview.emx* $\rightarrow$ *impedance_control_perfect*. Perfect impedance control

As now, it will receive a point for reference tracking, and instead of perfectly trying to follow the reference signal, it will perfectly mimic the behaviour of the virtual spring-damper of fig. 4.

2.1.5 Cartesian Impedance Control

The virtual spring is in Cartesian coordinates. In order to properly implement Cartesian Impedance Control, we must calculate the forward kinematics of the system. That is, we should find the Cartesian coordinates of the end-effector, given the measurable angles of our joints. One such way to find the coordinates of the end-effector is as follows:

$$\begin{aligned} r_{xy} &= L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2), \\ x &= r_{xy} \cos(\varphi), \quad y = r_{xy} \sin(\varphi), \quad z = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2). \end{aligned} \quad (6)$$

Here, r_{xy} is the radius of the distance of the end-effector after projecting it on the xy -plane, and $\varphi, \theta_1, \theta_2$ are the angles of Joint 1, 2 and 3, respectively.

We can then find the desired force by taking the difference between the desired and measured position and applying this on a virtual spring-damper as in fig. 5. The actuators, however, are rotational. Therefore, we have to use the transpose of the Jacobian to translate the desired forces into desired actuator torques.

Letting $s_\varphi = \sin(\varphi), c_\varphi = \cos(\varphi), s_1 = \sin(\theta_1), c_1 = \cos(\theta_1), s_2 = \sin(\theta_2)$ and $c_2 = \cos(\theta_2)$, the Jacobian is given as

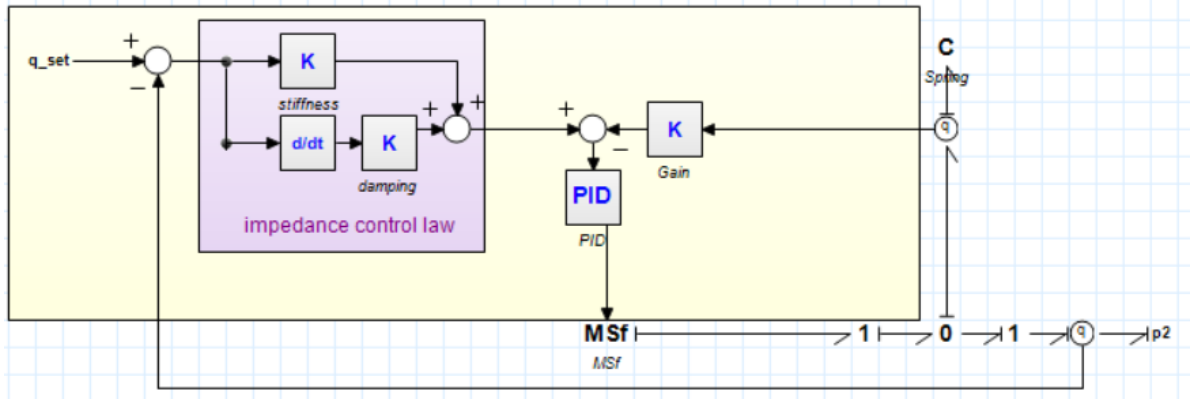
$$J = \frac{\partial(x, y, z)}{\partial(\varphi, \theta_1, \theta_2)} = \begin{bmatrix} -s_\varphi(L_1 s_1 + L_2 s_{12}) & c_\varphi(L_1 c_1 + L_2 c_{12}) & L_2 c_\varphi c_{12} \\ c_\varphi(L_1 s_1 + L_2 s_{12}) & s_\varphi(L_1 c_1 + L_2 c_{12}) & L_2 s_\varphi c_{12} \\ 0 & -(L_1 s_1 + L_2 s_{12}) & -L_2 s_{12} \end{bmatrix}. \quad (7)$$

The desired torques for our impedance controller are then found using the desired forces F_d as

$$\tau_d = J^T F_d. \quad (8)$$

2.1.6 Series Elastic Actuators (SEAs)

One actuator that is appropriate for Impedance Control is the SEA, which is illustrated below:

Figure 6: *G06_impedance-overview.emx* $\rightarrow$ SEA. Ideal SEA

It can be seen that first, the desired force output is found from a virtual implementation of the desired spring-damper. A physical spring is then attached in series after the actuator, making the actuator behave elastically to disturbances (hence the name series elastic), but it also allows the torques to be determined by measuring the angles. That is, given some known spring constant C , one can easily find the torque in the spring as

$$\tau = Cq, \quad (9)$$

with q the deformation of the spring and C the spring constant of the physical spring in the SEA. The SEA in fig. 5 is still idealised. It can perfectly mimic the desired spring-damper by picking a large proportional gain.

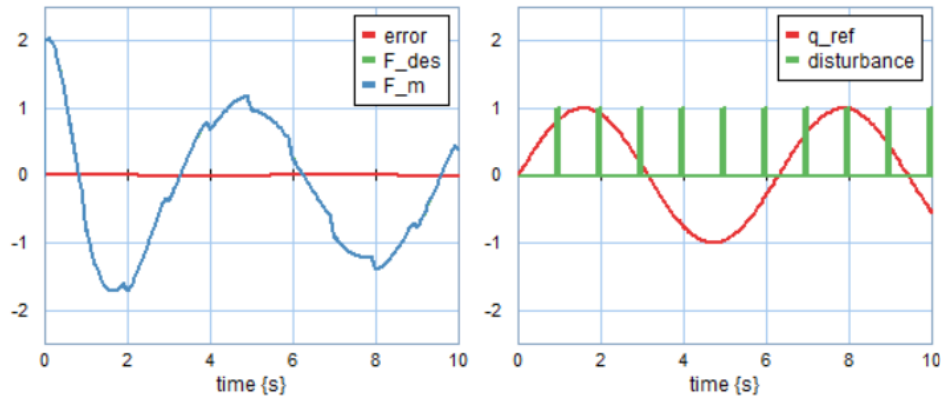


Figure 7: Ideal SEA behaviour matches desired torque exactly.

In order to decide the SEA stiffnesses, we use the following relation:

$$\omega_{sea} = \sqrt{\frac{K_{sea}}{I_{load}}} \implies K_{sea} = I_{load} \cdot \omega_{sea}^2 \quad (10)$$

Here we determine ω_{sea} by first defining minimum trajectories and completion times for each of the joints:

- joint 1: Complete half a rotation (180°) in 5 seconds (0.1Hz or 0.63 rad/s)

- joint 2: Complete a quarter rotation (90°) in 2.5 seconds (0.1Hz or 0.63 rad/s)
- joint 3 (connected to end-effector): Complete half a rotation (180°) in 2 seconds (0.5 Hz or 3.14 rad/s)

After deciding on this, we determine the bandwidth for the system with SEA, which should be at least 10 times higher than the determined trajectory frequencies for accurate motion tracking to avoid having a part of the resonance peak inside the control bandwidth.

To determine the SEA parameters accordingly, the above angular velocities are multiplied by 10, and a bit of margin is added. These angular velocities and the moments of inertia from table 7 are inserted in eq. (10) to find the minimum spring constants for each of the SEAs. These calculated SEA stiffnesses were further tuned in the model to be as low as possible while still having stable control and accurate motion tracking. The lowest possible stiffness was used for this as it ensures optimal shock absorbance and safety, which is what the SEA was implemented for.

The results are as follows:

Joint	ω_{min}	$\omega_{SEA} \approx 10\omega_{min}$	Min. spring constant	Tuned spring constant
Joint 1	0.63 [rad/s]	8 [rad/s]	933 [N.m/rad]	1500 [N.m/rad]
Joint 2	0.63 [rad/s]	8 [rad/s]	1020 [N.m/rad]	1500 [N.m/rad]
Joint 3	3.14 [rad/s]	32 [rad/s]	473 [N.m/rad]	600 [N.m/rad]

Table 8: Spring constants for the SEAs.

2.1.7 Torque Control

The torque control architecture serves as the intermediate control loop within the Series Elastic Actuators (SEAs). The overall actuation structure consists of three nested loops: the outer impedance control, the intermediate torque control, and the inner current control.

The desired reference torque (τ_d) is generated by the outer impedance controller and is combined with the feedforward gravity compensation signal. The measured torque (τ_m) can be accurately determined by sensing the deformation of the physical spring (q) and multiplying it by the tuned spring stiffness (K_{sea}), yielding $\tau_m = K_{sea}q$. This then allows for feedback on the reference signal to apply control to.

The error between the desired torque and the measured torque is fed into a PI controller. The output of this PI torque controller dictates the desired electrical current, which subsequently serves as the reference setpoint for the inner current control loop detailed in Section 3.5.2.

2.1.8 Gravity compensation

Gravity compensation is applied to our controller to greatly improve its performance. The effects of gravity have already been found in table 3. These are precisely the torques that are fed to the joints in the first column using feedforward to cancel the effects of gravity.

2.2 Model overview

Our full continuous model can be seen in figure 8 below:

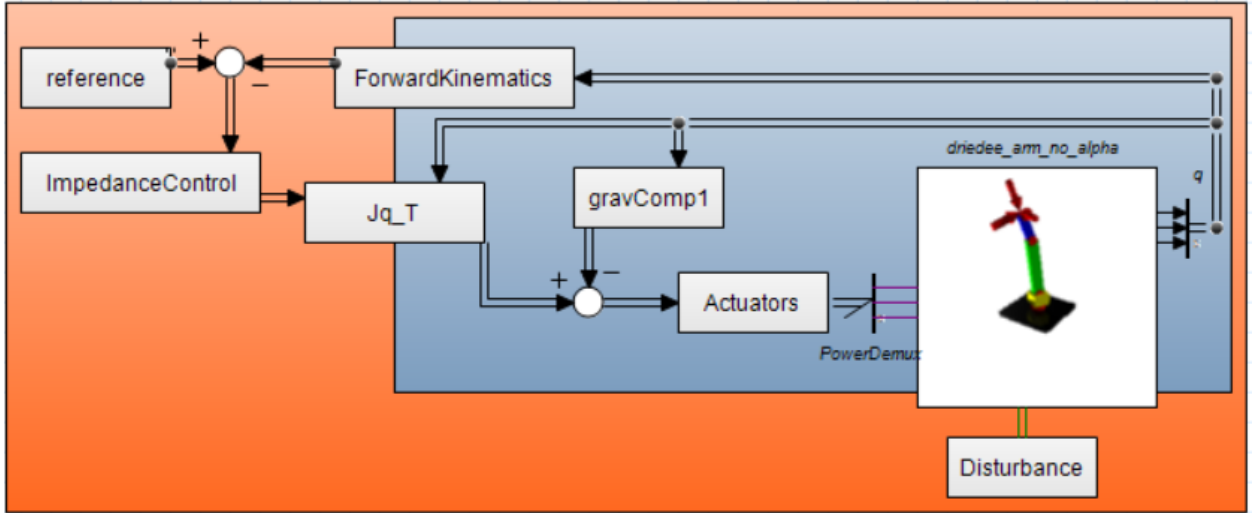


Figure 8: Continuous model overview, taken from *G06_model-overview*.

Here, from top to bottom, left to right, we have the following:

1. *reference*: coordinates (x_d, y_d, z_d) for the robotic arm end-effector to track.
2. *ForwardKinematics*: this block translates the robotic arm angles $(\varphi, \theta_1, \theta_2) \rightarrow (x, y, z)$ using eq. (6).
3. *ImpedanceControl*: The virtual spring-damper as in fig. 5.
4. *Jq_T*: The transpose of the Jacobian matrix as in eq. (7).
5. *gravComp*: Feedforward cancellation of gravity using the terms of table 3.
6. *Actuators*: An implementation of Joint 1, 2 and 3. This can be either:
 - (a) Perfect effort sources, as to model fig. 5,
 - (b) Ideal SEAs, as to model fig. 6,
 - (c) Realistic motor-gearbox combinations of section 2.1.2.
7. *driedee_arm_no_alpha*: an implementation of our robotic arm using the 3D Mechanics Editor of 20-SIM.

The full discrete model can be seen in figure 9. It is important to note that the only difference between the continuous and discrete model are the sampling and hold blocks, discretized controllers, differentiate and integrate functions. The results appeared to be exactly the same for both models. Therefore, due to the small model differences and large simulation time differences, the continuous model was used for most screenshots and simulations, unless explicitly specified.

The other difference with the continuous model can be seen inside one of the SEAs in the SEA1 submodel. The discrete SEA submodels are all visually the same and one can be seen in figure 10.

2.2.1 Modelling choices

To realistically model a physical implementation of such a SEA, we should replace the perfect flow source in fig. 6 with a realistic motor. Our realistic motor implementation for a SEA is as follows:

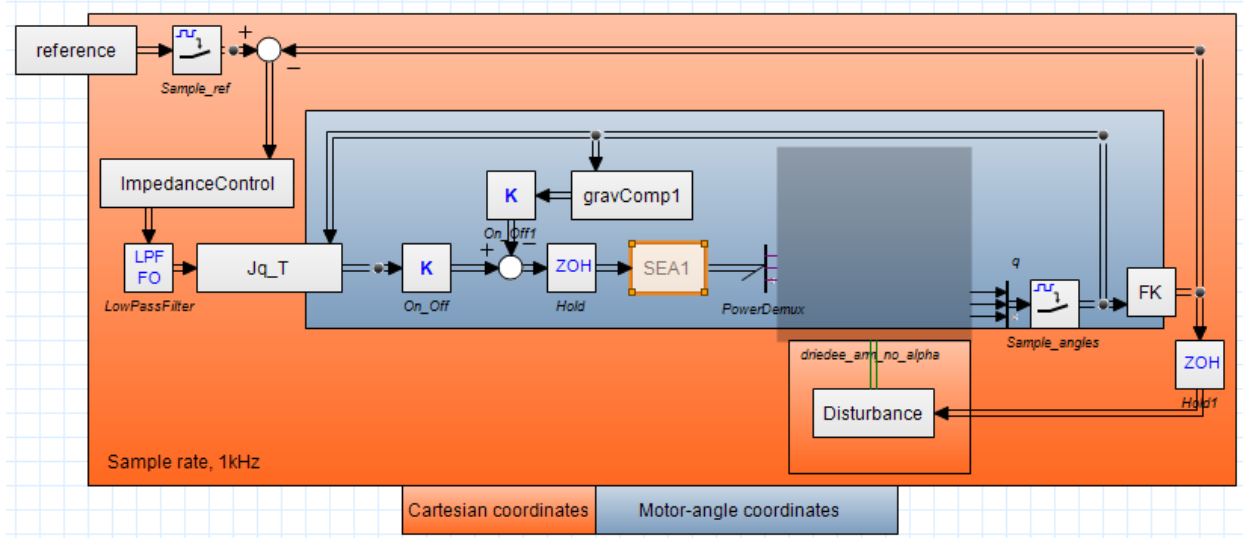


Figure 9: Complete discrete model overview including sampling rate of the outer loop, taken from *G06_final_discrete*.

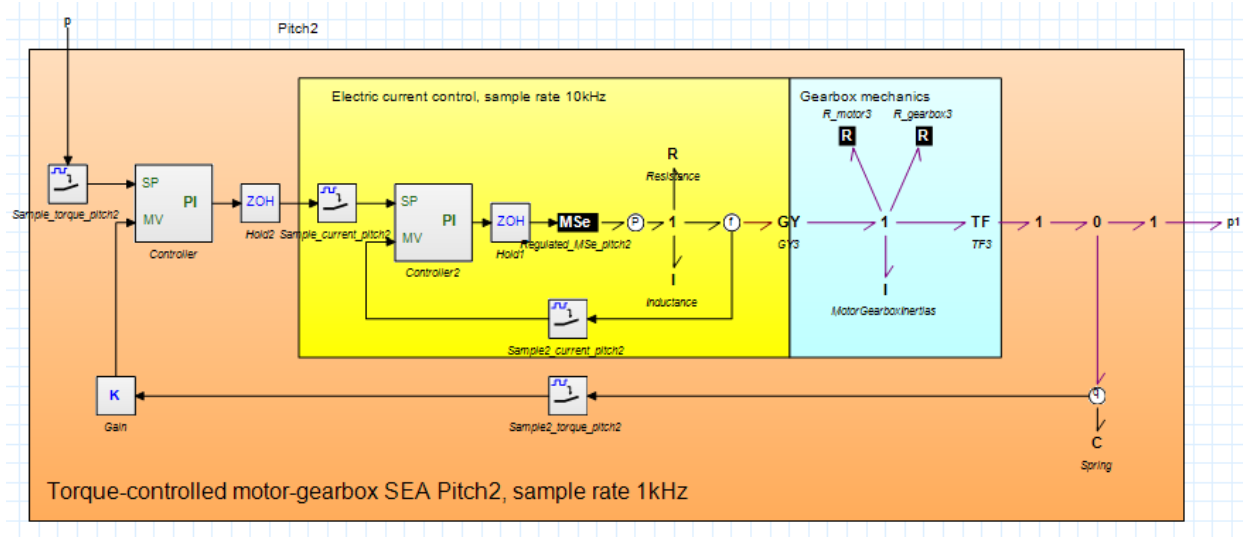


Figure 10: SEA1 discrete SEA overview with sampling rates of torque and current control. *G06_final_discrete* → *SEA1* → *SEAPitch2*.

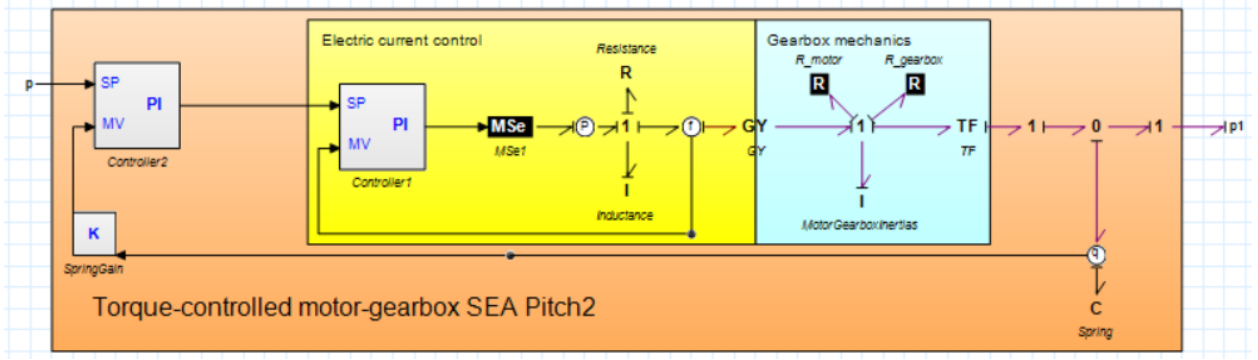


Figure 11: Motor-gearbox implementation of a SEA.

G06_final-SEAs-tuned-mooi-current.emx $\rightarrow$ *SEA1* $\rightarrow$ *SEAPitch2*

In the left part, a reference torque enters the motor. This is translated to an electric signal by the current controller, which in turn is translated by a Gyrator to a mechanical signal. The motor is equipped with a gearbox. We explicitly model the resistance, inductance, friction, and inertia of the relevant components.

Inside of this motor-gearbox system, we have customised the PI-controllers, the modulated effort source and both frictional elements.

The controllers are simple implementations of PI control:

```

1 // Created by C. Kleijn, 22 Sep 1999
2 parameters
3   real global pitch2_K;
4   real global pitch2_ki;
5 variables
6   real error;
7   real hidden uP,uI;
8 equations
9   error = SP - MV;
10  uP = pitch2_K * error;
11  uI = int (error * pitch2_ki);
12  output = uP + uI;

```

Listing 1: Implementation of PI torque control.

G06_final-SEAs-tuned-mooi-current.emx $\rightarrow$ *SEA1* $\rightarrow$ *SEAPitch2* $\rightarrow$ *Controller2*

The effort source is designed to cap the outgoing effort to 48 V, and the total outgoing power is capped to 480 W, as per the table 4. The code for the effort source is as follows:

```

1 parameters
2   real P_max = 480.0 {W}; // Maximum allowed continuous power 480
3   real V_max = 48.0 {V}; // Maximum physical supply voltage
4   real epsilon = 0.001 {A}; // Threshold to prevent division by zero
5
6 variables
7   real v_request {V};
8   real i_actual {A};
9   real v_power_limit {V};
10  real v_final_limit {V};
11
12 equations
13  // 1. Read the requested voltage from the PI controller signal and current
    flowing through the motor.

```

```

14  v_request = effort;
15  i_actual = p.f;
16
17  // 2. Calculate the voltage ceiling dictated by the 480W power limit
18  v_power_limit = P_max / (abs(i_actual) + epsilon);
19
20  // 3. The actual voltage ceiling cannot exceed the physical 48V power supply
21  if v_power_limit > V_max then
22      v_final_limit = V_max;
23  else
24      v_final_limit = v_power_limit;
25  end;
26
27  // 4. Apply the limits using a nested if-else to avoid syntax errors
28  if v_request > v_final_limit then
29      p.e = v_final_limit;
30  else
31      if v_request < -v_final_limit then
32          p.e = -v_final_limit;
33      else
34          p.e = v_request;
35      end;
36  end;

```

Listing 2: Limited effort source.

G06_final-SEAs-tuned-mooi-current.emx → *SEA1* → *SEAPitch2* → *MSe1*

The total motor friction was calculated with the knowledge that when the motor is operating at the maximum speed, the total torque being put into the system is equal to the total friction in the motor. The formula that was used for this is as follows:

$$R_{fric} = I_{nl} \cdot K_t, \quad (11)$$

where I_{nl} is the current at the no load speed and K_t is the torque constant of the motor. Substituting the appropriate motor values, which can be found in their respective manuals, the total friction at no load speed for each motor was found to be:

Motor	I_{nl}	K_t	R_{fric}
1 and 3	0.482 [A]	0.0705 [N.m/A]	0.034 [N.m]
2	0.815 [A]	0.124 [N.m/A]	0.101 [N.m]

Table 9: Friction inside the motors.

This total friction at no load speed contains both the coulomb (static friction) and viscous friction (scales with speed) terms, for the motor we assume this ratio to be around 20% coulomb and 80% viscous. As the bearings in the motor have a relatively low friction to start moving (coulomb friction), compared to the viscous term. For the viscous term, the friction scales with velocity divided by the no load speed, as the total friction was calculated at the no load speed, which would give maximum viscous friction.

The final formula used in 20-sim for the motor friction is:

```

1  parameters
2  real r = 0.034; // I_nl * K_t (no load torque = friction) calculated
   analytically
3  real no_load_speed = 6420 * (twopi/60); // rad/s

```

```

4   real g = 500; // scaling factor for sign approximation, make smaller if slow
      performance
5
6 equations
7   // Assuming a ratio of 20-80 coulomb-viscous friction (most viscous due to
      little dry friction from bearings)
8   p.e = 0.8 * r * p.f/no_load_speed + 0.2 * r * tanh(g*p.f);

```

Listing 3: Motor friction for motor 1.

G06_final-SEAs-tuned-mooi-current.emx $\rightarrow$ *SEA1* $\rightarrow$ *SEAPitch2* $\rightarrow$ *R_motor*

The total gearbox friction is modelled in a similar fashion as the motor friction, but in a 80-20% ratio viscous-Coulomb friction:

```

1 parameters
2   real r = 0.0226; // tot = 0.0226, found from fixed torque (max nominal torque
      because we expect highest efficiency there) and fixed speed
3   real g = 500; // scaling for sign approximation, make smaller if slow
      performance
4
5 equations
6   // Assuming a ratio of 80-20 coulomb-viscous friction (most coulomb due to dry
      friction from gears (non-speed dependent))
7   p.e = 0.2 * r * p.f + 0.8 * r * tanh(g*p.f);

```

Listing 4: Motor friction for motor 1.

G06_final-SEAs-tuned-mooi-current.emx $\rightarrow$ *SEA1* $\rightarrow$ *SEAPitch2* $\rightarrow$ *R_gearbox*

Most of the inertias and resistances of fig. 11 can be found in table 4 and 5. We do add 0.3Ω resistance to motor 1 and 3 and 0.03Ω to the electric resistance inside motor 2, (R_{elec} in fig. 11) to account for dissipation in the wires and motor driver. Motor 2 should have a motor driver with less resistance as the resistance in the motor itself is significantly lower than in motor 1 and 3.

3 Simulation

In order to find the frictions inside the gearboxes, tune our controllers and verify our models, we performed the following simulations:

1. **Base model simulations**, without actuation but with a model for friction inside the joints, to check whether the base model behaves as a double pendulum.
2. **Perfect Impedance control simulations with gravity compensation**,
 - (a) to verify whether the system can be controlled accurately using Impedance Control, and
 - (b) to verify whether we can indeed make the system react to disturbances as we described in section 1.2.
3. **Gravity compensation verification**, by having the perfect impedance control with gravity compensation steer the arm to several different configurations using ideal motors, and then checking whether the arm stays in place when the Impedance Control is turned off.
4. **Perfect SEAs for Impedance Control** to verify the effectiveness of SEAs in the model.
5. **Isolated motor simulations**
 - (a) To find and validate the current control of each motor, and
 - (b) to find the friction inside the gearboxes.
6. **Ordered realistic motor-gearbox tuning**, using the argument from fig. 2: first, tune a realistic motor-gearbox implementation for Joint 3, then 2, then 1. These will be tuned in their operational workspace, and preceding realistic implementations will stay in place during the tuning of every new realistic implementation.
7. **Discrete-time realistic motor-gearbox SEAs for Impedance Control** to finalise the modelling and simulation of a realistically actuated robotic arm.

3.1 Base model simulations

We will now verify the model for the actual robotic arm. The model of fig. 12 was used for these simulations.

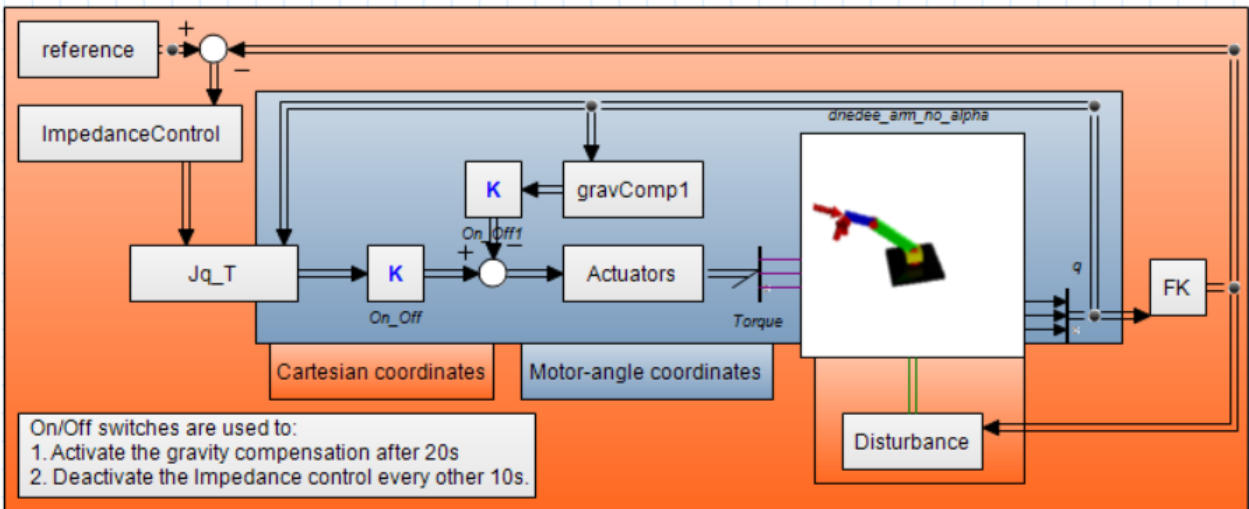


Figure 12: *G06_robot-arm-model-verification.emx*

This model is mostly the same as fig. 8, with the differences that some On/Off switches have been included, and the reference and disturbance blocks are equipped with tools to plot the reference and disturbance signals in the animation. Furthermore, ForwardKinematics has been abbreviated to FK.

In order to verify basic validity of our model, we turn off the actuators and let the arm fall to a halt. We will remove the limiting angles of Joint 2 to verify that the arm will indeed fall down as a double pendulum would. The initial angles will be set to $(\varphi, \theta_1, \theta_2) = (0, 45, 20)$ [deg]. We will let the simulation continue like this for 20 seconds without disturbances.

The friction model in the joints distributes a friction of 4.0 [N.m] for Joint 1 and 3 and of 8.0 [N.m] for Joint 2 using a 20-80% split over viscous and Coulomb friction, respectively. That is, a nominal friction r in the joint will result in a relation

$$\tau_R = 0.2r\omega + 0.8r \tanh(g * \omega), \quad (12)$$

with ω the angular velocity of the joint, and $g = 500$ some scaling factor for the Coulomb-friction approximation.

The results of this simulation are combined with those of section 3.2 and section 3.3 into one video.

3.2 Perfect Impedance Control with gravity compensation

After the first 20 seconds to verify the basic validity of our robotic arm, we will switch the actuators on, as well as the gravity compensation. The end-effector should track the reference signal, which is made up of the following effects:

1. A radius pulse, expanding and shrinking the radius on which the desired position can be from the origin.
2. A vertical sweep from a height of 0.5 m to the maximum height based on the above radius pulse.
3. To find the (x, y) coordinates of the desired trajectory, first determine the allowed horizontal cross-section. This is the intersection of a sphere with a radius from (1.) and a horizontal plane with a height from (2.).
4. The reference trajectory then moves over (x, y) on this intersecting circle as a regular sinusoid.

The result of this simulation is combined with that of section 3.1 and section 3.3 into one video.

3.3 Gravity compensation verification

In the simulation of the previous two subsections, the Impedance Controller is now turned off every 10 seconds for a duration of 10 seconds, while the gravity compensation is kept on. This provides a wide range of positions from which the gravity compensation can be checked.

The result can be found [online](#).

Interestingly, the modulo operator in 20-SIM does not turn the Impedance Control on or off at exactly the right moments. We are not sure what causes this, but it does not influence the validity of the above results.

3.4 Perfect SEAs for Impedance Control

We refer to *G06_final-perfect-SEAs.emx* for running the relevant simulations.

Now the perfect actuators are replaced one by one by perfect SEAs, each with a spring constant from table 8. First, we replace Joint 3 by an ideal SEA and optimise its torque control, then Joint 2, and then Joint 1.

The optimisation was done using the Time domain toolbox in 20-SIM. The whole PI groups were optimised based on their integral square error for 15 seconds of active Impedance-control time. The results can be seen in table 10 below:

SEA	k_p	k_i
Joint 3	0.05	0.05
Joint 2	0.5	0.05
Joint 1	10.0	0.05635

Table 10: Perfect SEAs PI parameters

Note that these are the nominal values that we found to work. The control toolbox was unable to significantly improve on them.

Interestingly, the gravity compensation on its own is no longer sufficient to keep the robotic arms aloft. When the Impedance reference signal is set to 0, the arms will slowly start to fall down. Reasons for this are investigated in the discussion.

Once again the resulting animation can be found [online](#).

We also checked whether a constant disturbance would yield the desired deflection. For this, we set the reference point to (0.5, 0, 1) and pulled on the end-effector with a constant force to measure the spring constant. Using a constant force of 150 [N] and checking for a deflection of 0.5 [m] is inaccurate, as this deflection is too large for the relevant workspace. That is why we used a small force of 25 [N] and checked whether the resulting deflection can be attributed to a spring constant of 300 [N.m].

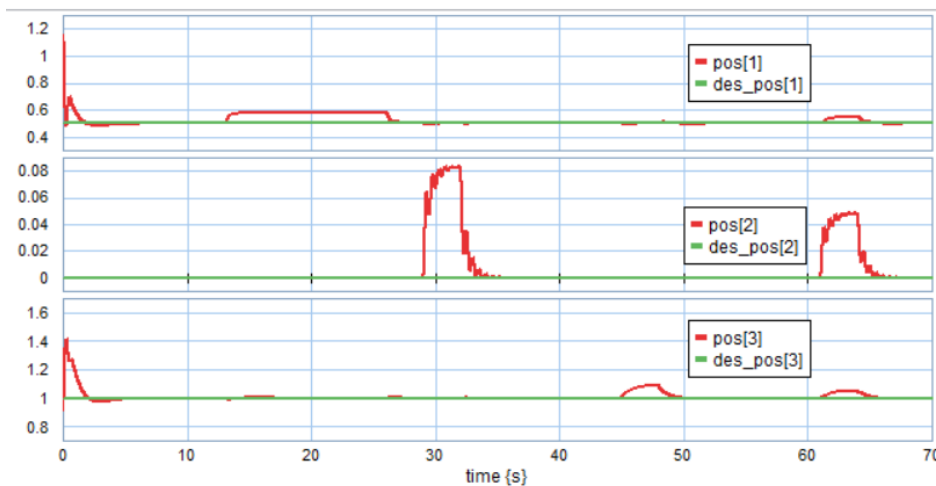


Figure 13: Resulting deflections from constant disturbances.

The effects can most clearly be seen (fig. 13) in the y -direction (middle graph). Indeed, the robotic

arm reacts like a spring that is not too stiff for human interactions. The spring constant can most accurately be measured by checking the difference in x over a longer period of time. There, a deflection of 0.085 [m] was found. The resulting spring constant is then

$$C = \frac{25[N]}{0.085[m]} \approx 294.12[N.m^{-1}]. \quad (13)$$

This is very close to the desired spring constant of 300 [N.m⁻¹] from our introduction.

3.5 Isolated motor simulations

All three series elastic actuators of the joints consist of two nested control loops. The input is the desired torque, which comes from the impedance controller. Then, a torque controller computes the required current. Lastly, a current controller is used to control the motors.

Before the control loops can be tuned, the friction in the gearbox must be estimated.

3.5.1 Gearbox frictions

We computed the gearbox friction by looking at the maximum efficiency specified on the datasheets. It was reasoned that this maximum efficiency would occur at the maximum possible torque, with a relatively low velocity. The reason for this is that the viscous friction term is lowest here, while the coulomb friction is independent of speed and most easily overcome with higher torques. This assumption was verified by ramping the torque at a constant angular velocity, and plotting the efficiency, which indeed showed this relation between torque input and efficiency, as can be seen in figure 14.

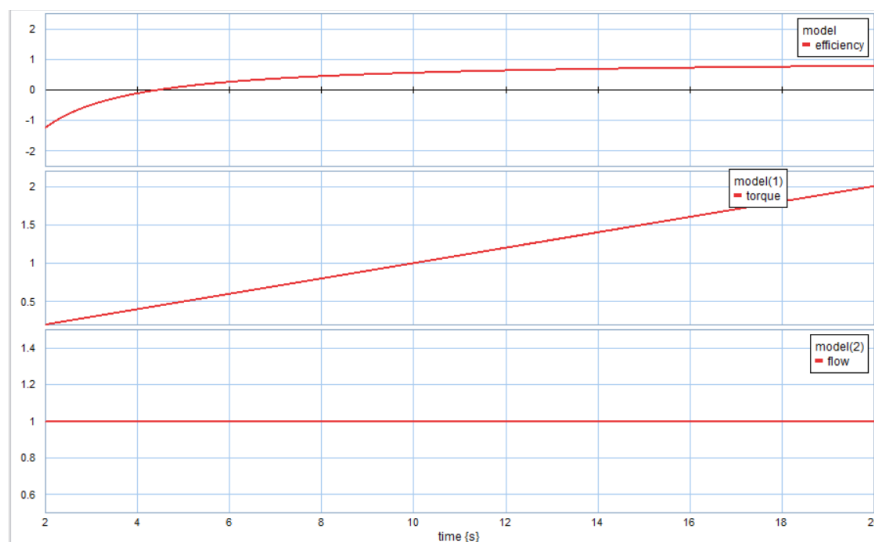


Figure 14: *G06_gearbox_friction.emx*, with “ramp” connected to MSe: Plot with the efficiency at a ramping torque input and constant angular velocity output.

Because of this, we used the nominal input torque (nominal output torque (from datasheet)/ reduction ratio) and a constant angular velocity of 1 rad/s to find the total friction in the gearbox. The friction was found by performing a parameter sweep at the highest possible torque until the friction term matched the maximum efficiency specified by the data sheet.

The efficiency and total friction for each gearbox can be seen in table 11.

Gearbox	Efficiency	Total friction
1, 3	92%	0.0226 Nm
2	70%	0.0267 Nm

Table 11: Efficiency and total friction per gearbox.

The total friction term for the gearbox is split into a ratio of 80% coulomb and 20% viscous friction. The reason for this assumption is that gearboxes generally have gears which have a relatively high friction to start moving (coulomb friction) compared to viscous friction.

A screenshot of the .emx model used for this can be seen in figure 15. The inertia of the gearbox may be neglected, as the fixed flow source on the output negates the effects of this inertia.

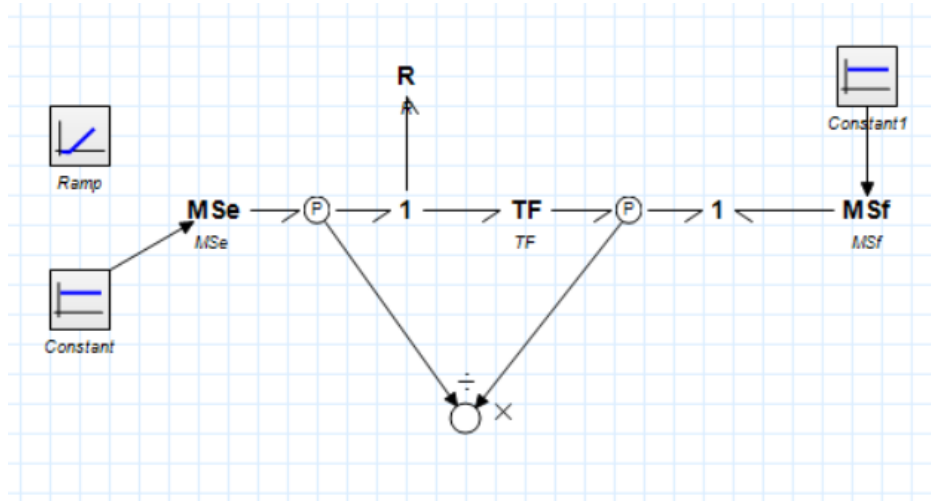
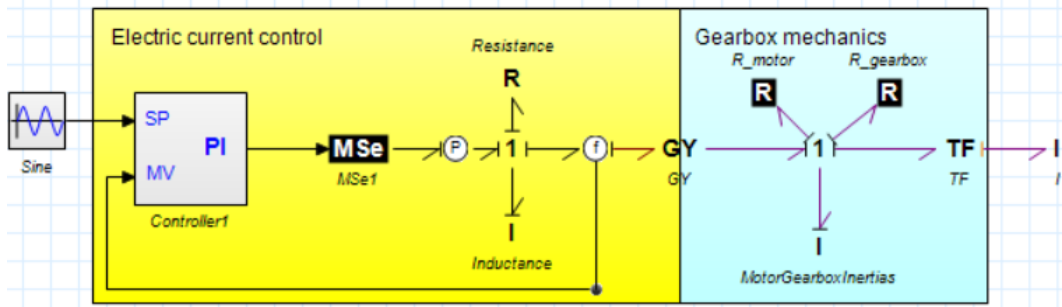


Figure 15: *G06_gearbox_friction.emx*: Model used to parameter sweep the gearbox friction to match the maximum efficiency.

3.5.2 Current control

In order to properly control the joint motors, a current controller is required. A simple PI controller was chosen, as the integral term is needed to compensate for the steady state error. Theoretically, the gains of the controller can be infinite to get the fastest response. In reality, the motor can only handle 48V, which will be limited after the tuning. Therefore, a controller reaching steady state quickly without overshooting is desired.

Both motors first have their current controllers tuned in a separate model. These tuned current controllers are then verified in the complete model. The separated tuning model is the following:

Figure 16: *G06_current-tuner.emx*. Model for current tuning.

Here, we have simply used the relevant values that correspond to the motor-gearbox combination, and the load from table 7.

Maxon EC-4pole 32 (Yaw and Pitch2)

The model tries to follow a sinusoidal reference signal and is tuned using the time-domain optimisation toolbox of 20-SIM. The optimisation returns the following porportional and integral gain parameters:

$$k_p = 98.014 \quad k_i = 0.016$$

This PI controller yields the following result, with SP the setpoint and MV the measured value

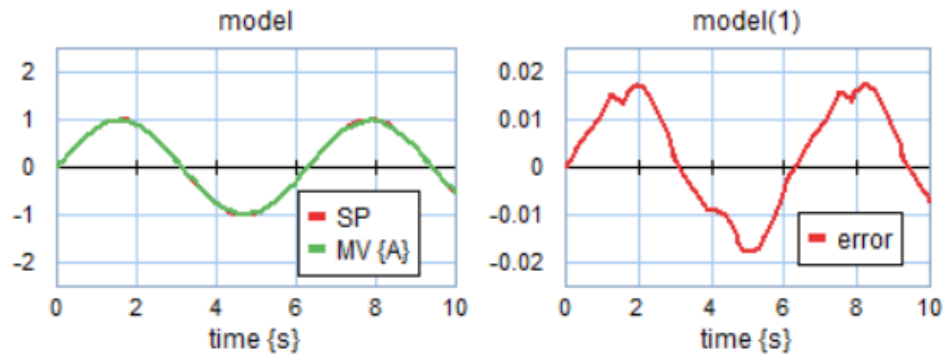


Figure 17: Current controller performance.

Clearly, this current control seems to be working well.

Maxon GPX 70 A (Pitch 1)

The same PI gains were found to work for the second motor as well, with the following result:

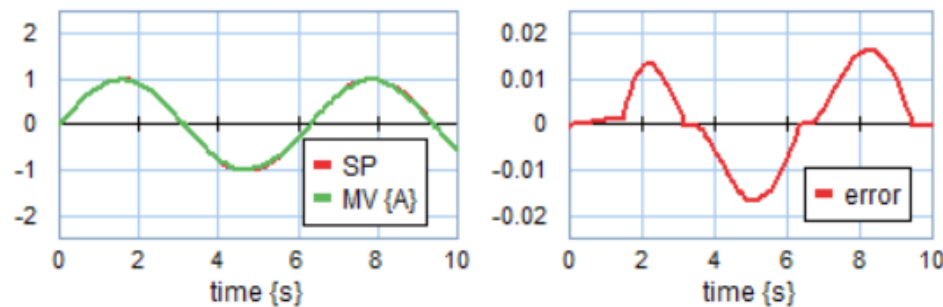


Figure 18: Second current controller performance

We decide to implement $k_p = 100$ and $k_i = 0.02$ in our current controllers.

3.6 Ordered realistic motor-gearbox tuning

As was stated before, we will implement realistic motor-gearbox combinations for our SEAs one by one. The model of fig. 19 was used for this purpose. It is nearly identical to fig. 12 and only differs in that it includes a low-pass filter after the virtual spring-damper and we have replaced SEA with SEA1, SEA2, and SEA3.

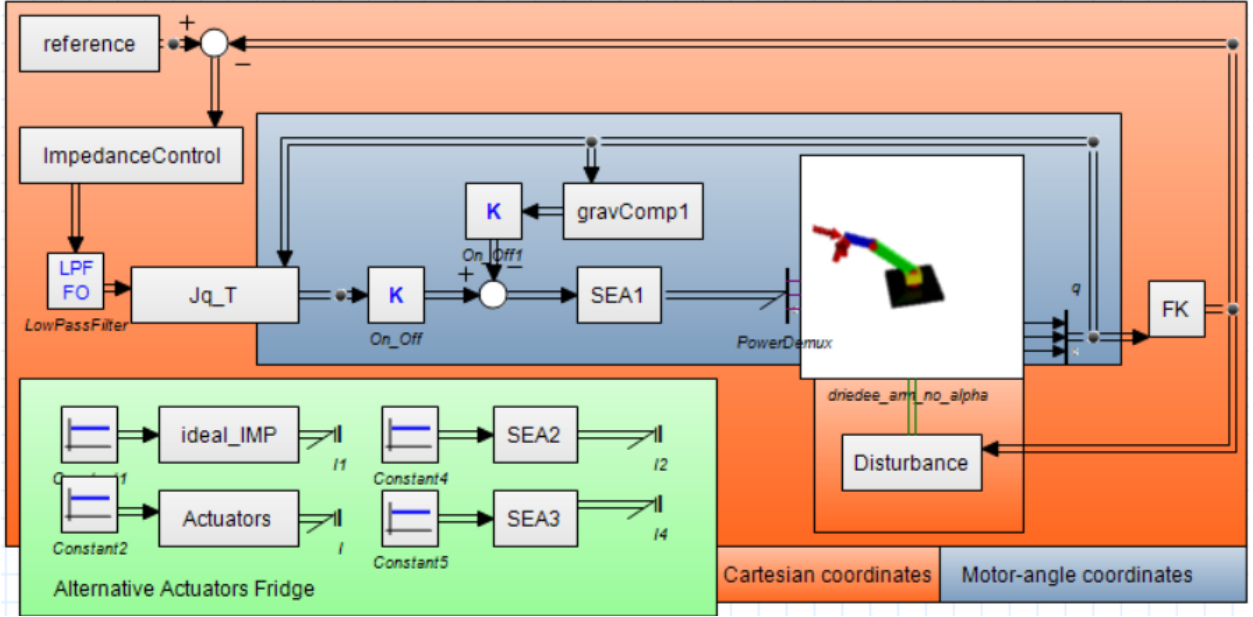


Figure 19: SEA tuning model.
G06_final-SEAs-tuned-mooi-current.emx

The low-pass filter was included to prevent very fast dynamics in the torque and current control, due to noise from the impedance controller.

The tuning of the torque control loop with realistic SEAs is done in exactly the same way as the tuning of the perfect SEAs, and for this approach, we refer to section 3.4. The final parameters found can be seen in table 12.

SEA	k_p	k_i
Joint 3	1	0.8
Joint 2	18.968	0.1897
Joint 1	31.76	1.048

Table 12: Realistic SEAs torque PI parameters

It appears most of the parameters translate well to realistic motors, except for Joint 3. We plot the behaviour of the torque controllers (top) and the current controllers (bottom) in the Yaw, Pitch1, and Pitch2 in figures 20 - 22. The result is as follows:

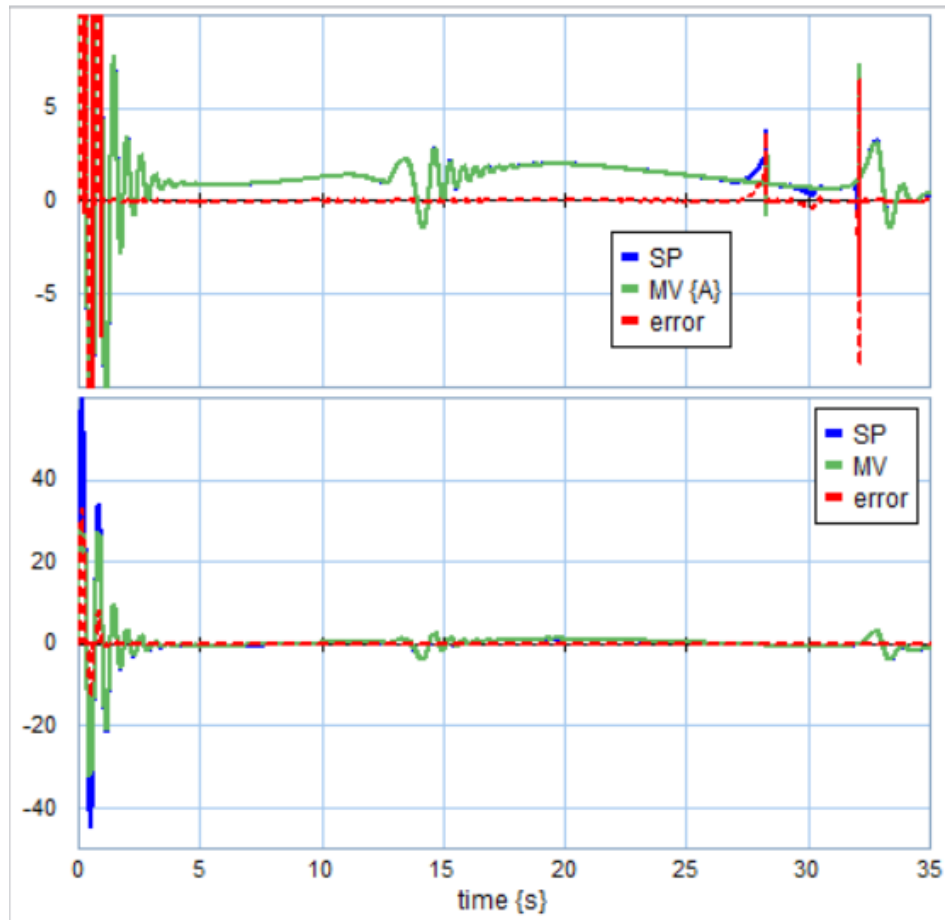


Figure 20: Control behaviour in the yaw

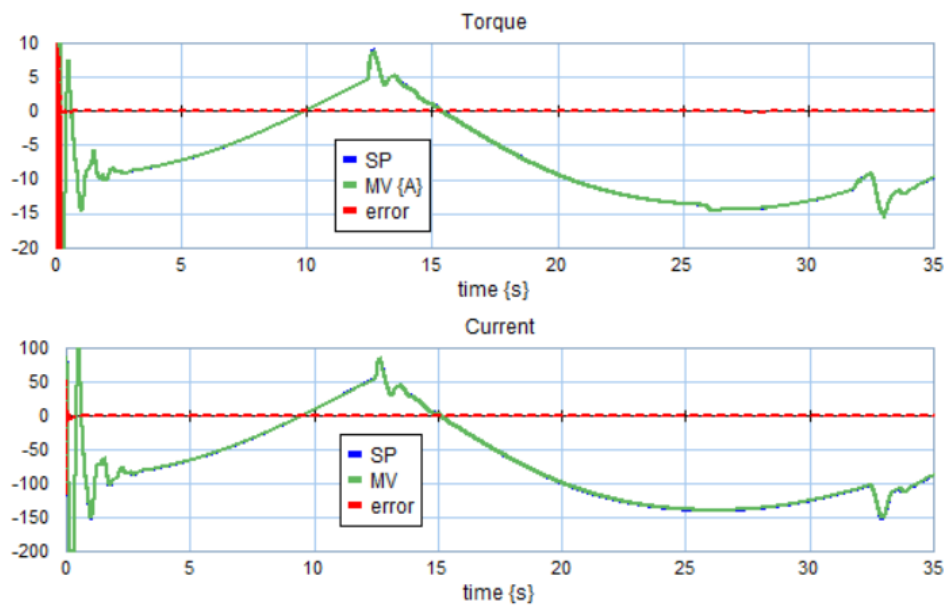


Figure 21: Control behaviour in pitch1

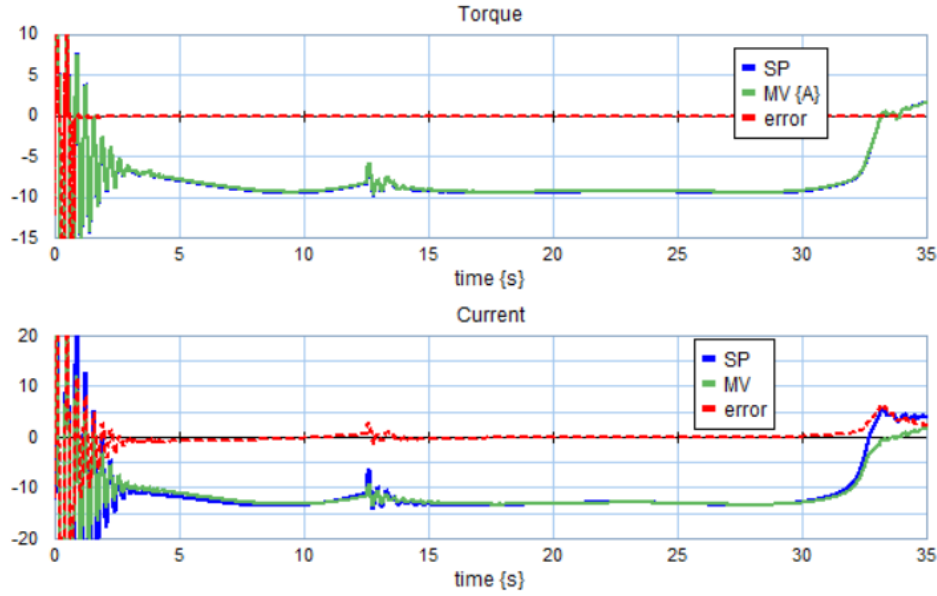


Figure 22: Control behaviour in pitch2

We can conclude that our current and torque controllers are working well in continuous time.

3.6.1 Continuous Impedance Control

To validate the outer Cartesian impedance control loop, an external disturbance was applied directly to the end-effector to simulate a collision or human interaction. The reference position is fixed for multiple configurations for this simulation as it shows the working of the disturbance best. The model used for this simulation is: *G06_cont_impedance_test*.

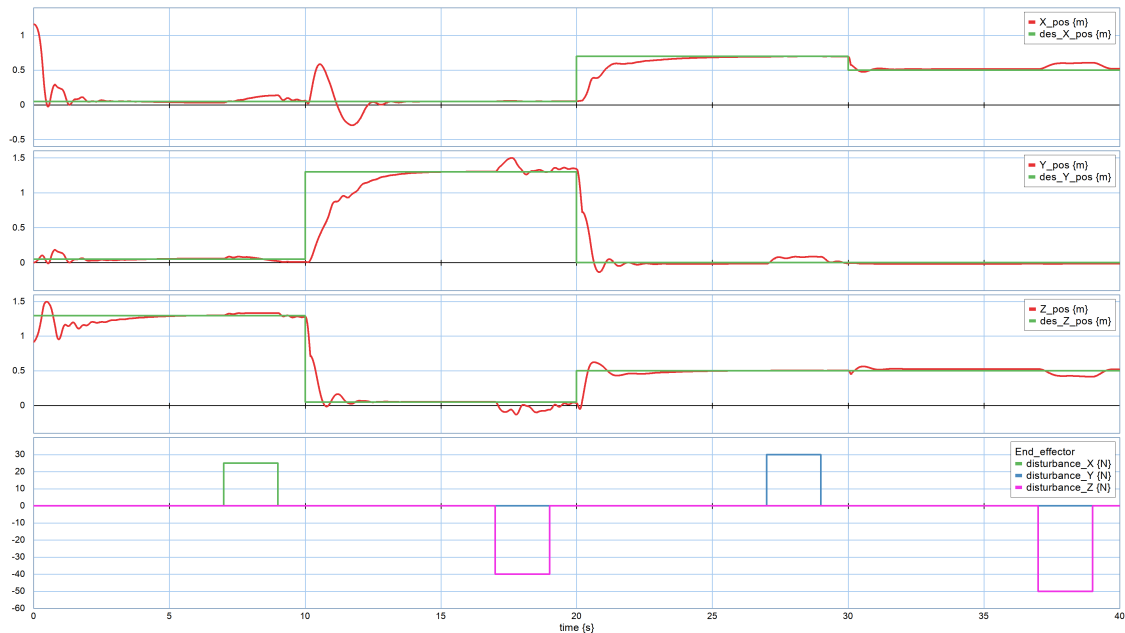


Figure 23: End effector position during disturbance

The disturbance force and the resulting positional deviation are plotted in fig. 23. The system behaves precisely as the designed virtual spring-damper, yielding to the disturbance to cap the

maximum interaction force, and smoothly returning to its reference trajectory once the disturbance is removed.

As our robot arm is relatively large and heavy, small disturbances only yield small positional deviations. Therefore larger disturbances were plotted and the deviations are indeed as expected. Even larger disturbances would also result in larger deviations.

Furthermore, we validate that we are still mimicking a spring with a damping constant of $300 \text{ [N.m}^{-1}\text{]}$. For this purpose, we fix the desired position in $(0.5, 0, 1)$ and apply once again a constant force of 25 N in the y -direction on the end-effector. This gives the following result:

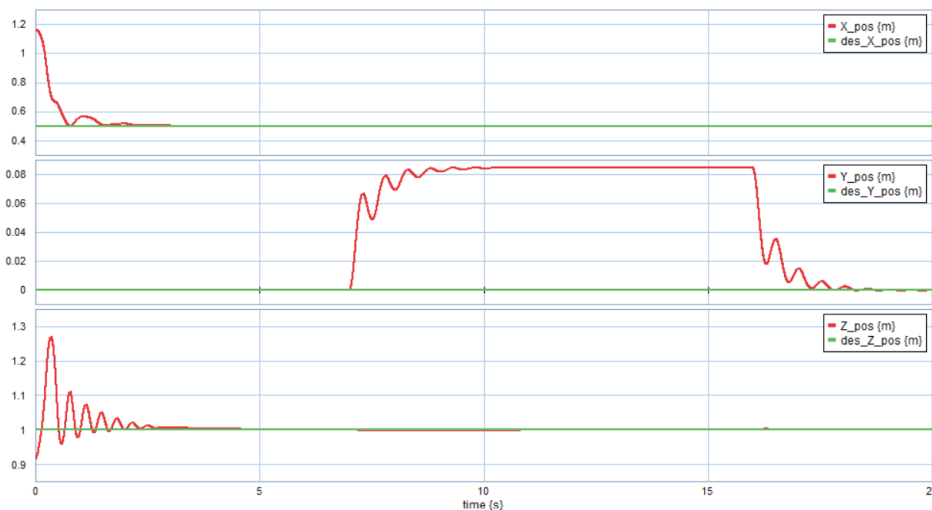


Figure 24: System response to constant disturbance

The deflection is about 0.0848 [m] . This yields an estimated spring constant of $294.811 \text{ [N.m}^{-1}\text{]}$, which is again very close to our desired $300 \text{ [N.m}^{-1}\text{]}$.

3.7 Discrete-time realistic motor-gearbox SEAs for Impedance Control

To transition our model closer to a physical, real-world implementation, the continuous controllers within the system were discretized. To accurately reflect hardware capabilities and ensure control stability, different sampling rates were chosen for the nested loops based on their dynamic requirements.

As illustrated in the system overviews (Figures 9 and 10), the inner electrical current control loop must react to the fastest dynamics of the system. Therefore, it is sampled at a high frequency of 10 kHz . The intermediate torque control and the outer Cartesian impedance control deal with slower mechanical dynamics and are appropriately sampled at 1 kHz . Zero-Order Hold (ZOH) blocks were implemented to bridge the discrete controllers with the continuous physical domains.

The chosen sampling rates correspond to real sensors and implementations.

3.7.1 Discrete Current Control

The current controller was then discretized with a sampling frequency of 10 kHz . As the electrical dynamics of the motor are the fastest in the system, this high sampling rate is necessary to maintain stability. The zero-order hold (ZOH) effects are negligible at this frequency, confirming that the digital implementation will accurately track current in the physical hardware.

It was however noticed that the current tracking did not perform as well with the same PI parameters as for the continuous domain, as can be seen in figure 25. This meant that retuning was necessary, sadly we did not have enough time to fully optimize these controller gains, but we did manage to get adequate results with trial and error. The final parameters found can be seen in table 13. More details about the results can be seen in section 3.7.2.

SEA	k_p	k_i
Joint 3	1	0.0003
Joint 2	0.25	0.000066
Joint 1	1	0.02

Table 13: Discrete current control PI parameters

3.7.2 Discrete Torque Control

The torque control loop was discretized at a lower sampling rate of 1 kHz, reflecting the slower mechanical dynamics compared to the electrical domain. In fig. 25, it can be seen that the overall position tracking performance remains robust, but one thing that was noticed when keeping the PI values the same as in the continuous case, is that there is significant oscillations in the output of the torque control (desired current). This can be seen in the lower part of the figure.

This showed that the control parameters were not optimized well for the discrete model. Which meant retuning the parameters in the current and torque PI blocks was necessary, sadly we did not have enough time to fully optimize these controller parameters but we did manage to get a good enough performance with trial and error.

The final parameters in the discrete current control PI block are listed in table 13 and the parameters in the discrete torque PI blocks are listed in table 14, which seems to have resolved the too aggressive controller response. The resulting reference tracking and current plot can be seen in figure 26.

SEA	k_p	k_i
Joint 3	9.802	0.294
Joint 2	19.798	0.1816
Joint 1	3.122	0.1031

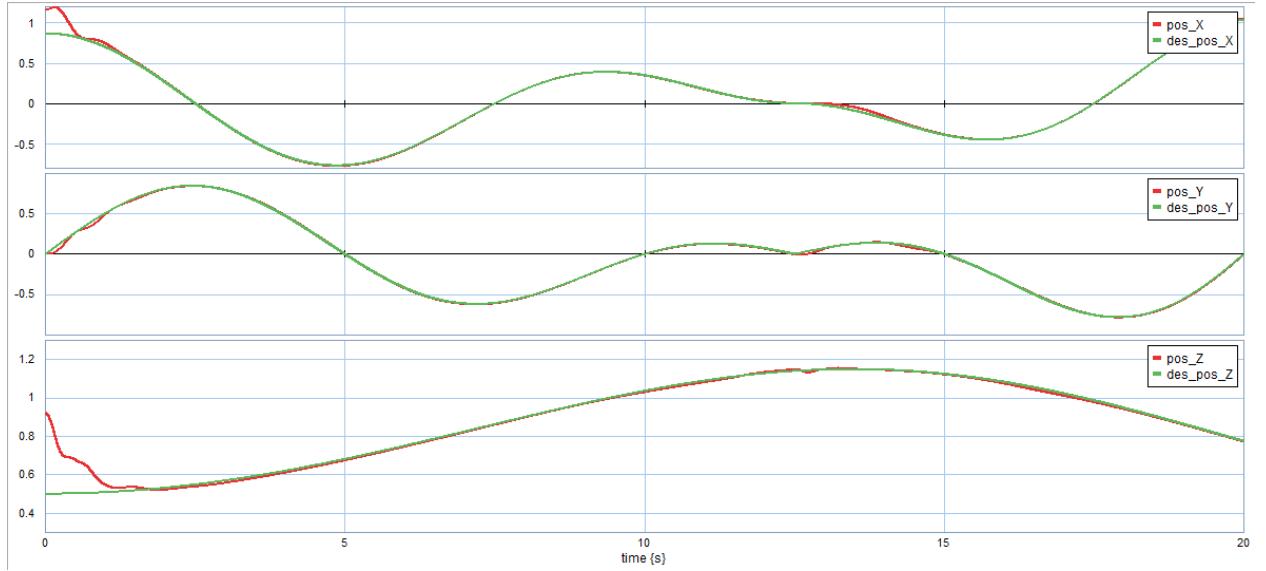
Table 14: Discrete torque control PI parameters

At these new PI values for joints, the system maintains stability, proving that 1 kHz is a sufficient sampling rate for the intermediate loop, although some additional tuning was needed for the conversion from continuous to discrete. The tracking performance appears the same for both cases.

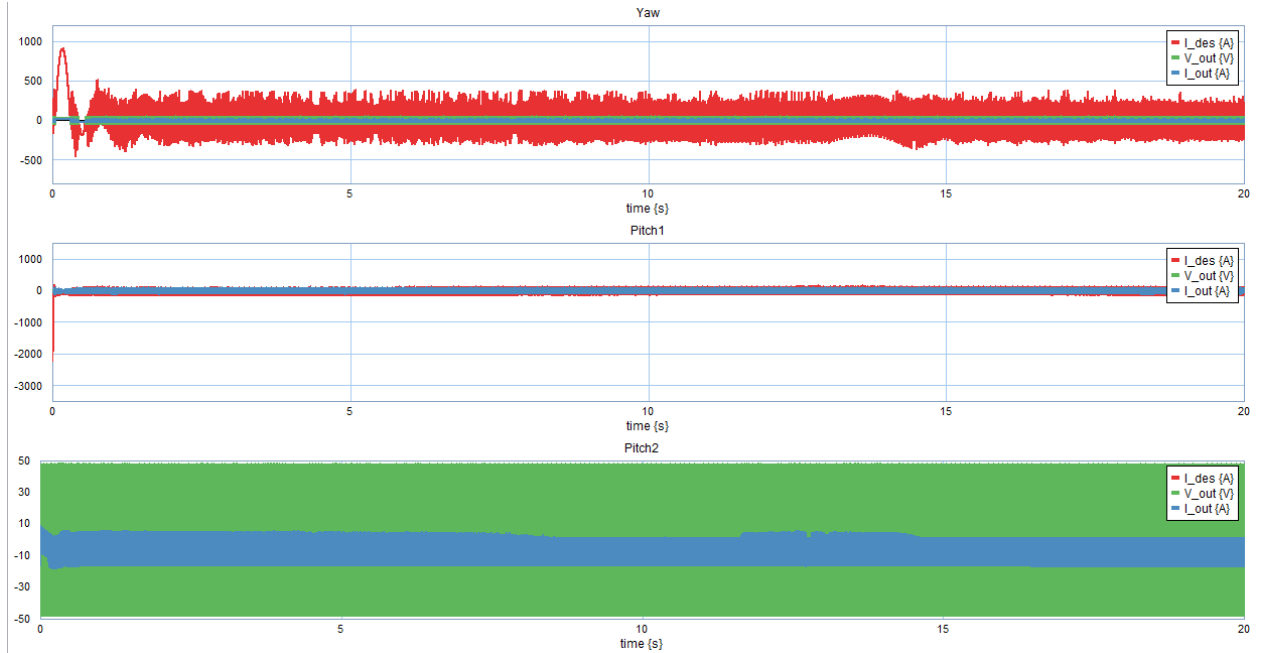
3.7.3 Discrete Impedance Control

Finally, the full discrete model was subjected to the same reference tracking and external disturbance at the end effector as for the impedance control in the continuous domain, to verify the digital impedance controller sampled at 1 kHz. The model used for this simulation is: *G06_disc_impedance_test*.

The used external disturbance and the resulting positions of the end effector can be seen in figure 27, the discrete system shows the same results as the continuous model (see figure 23) when it comes to the impedance control. The discrete system still behaves precisely as the designed virtual spring-damper when a force is applied at the end effector. Which is the exact same as was the case for the continuous system.



(a) Position tracking performance with same torque control PI values as continuous.

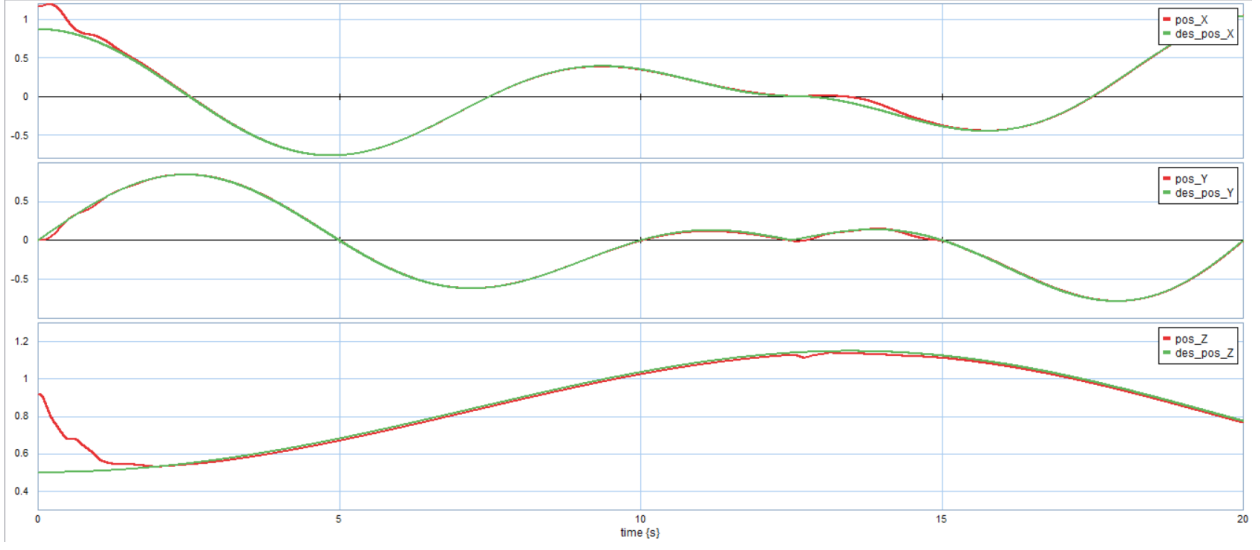


(b) Desired current, output voltage and output current plot with the same torque control PI values as continuous.

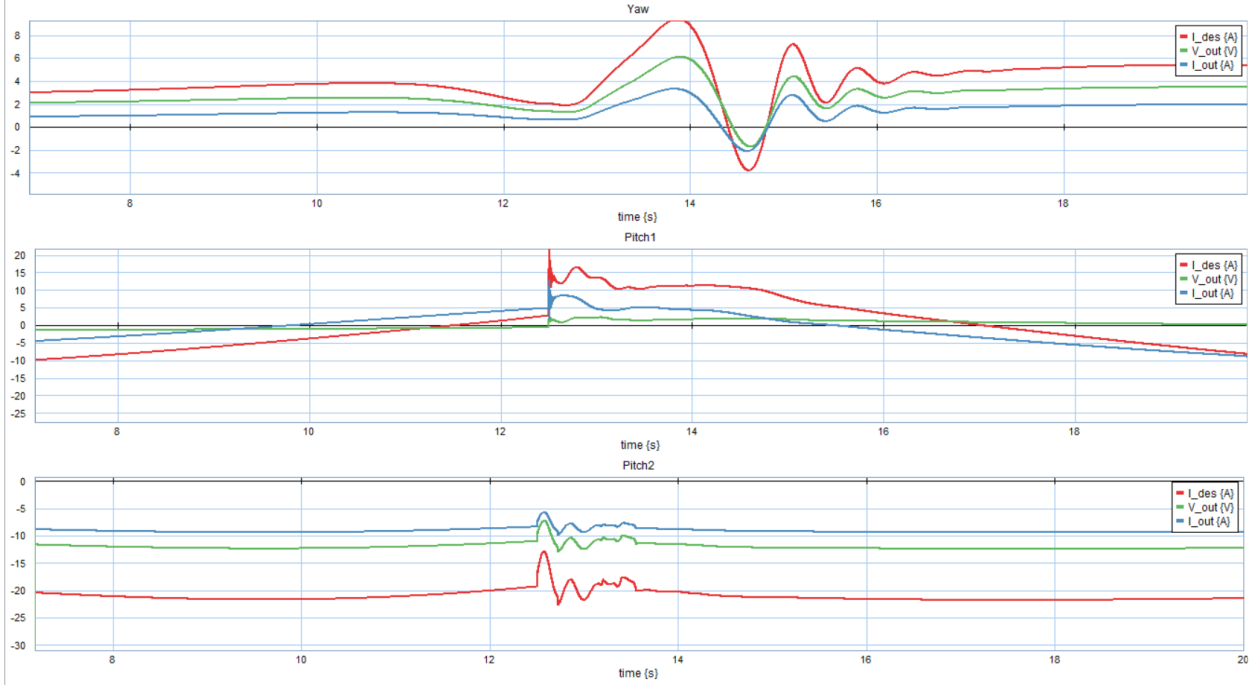
Figure 25: Figure with both the position tracking and desired and output current plots for a torque controller with the same PI values as the continuous torque and current controller.

G06_final_discrete.emx with disturbance turned OFF.

The resulting animation can be seen [online](#) (make sure to view from your utwente account if access is denied).



(a) Position tracking performance with changed torque control PI values for the joints compared to continuous.



(b) Desired current, output voltage and output current plot with changed torque control PI values for joints compared to continuous.

Figure 26: Figure with both the position tracking and desired and output current plots for a torque controller with changed torque control PI values for the joints compared to the continuous torque controller.

G06_final_discrete.emx with disturbance turned OFF.

4 Discussion & Conclusion

4.1 Discussion of modelling challenges

During the development and simulation of the robotic arm, several modelling and control challenges were encountered that required iterative refinement.

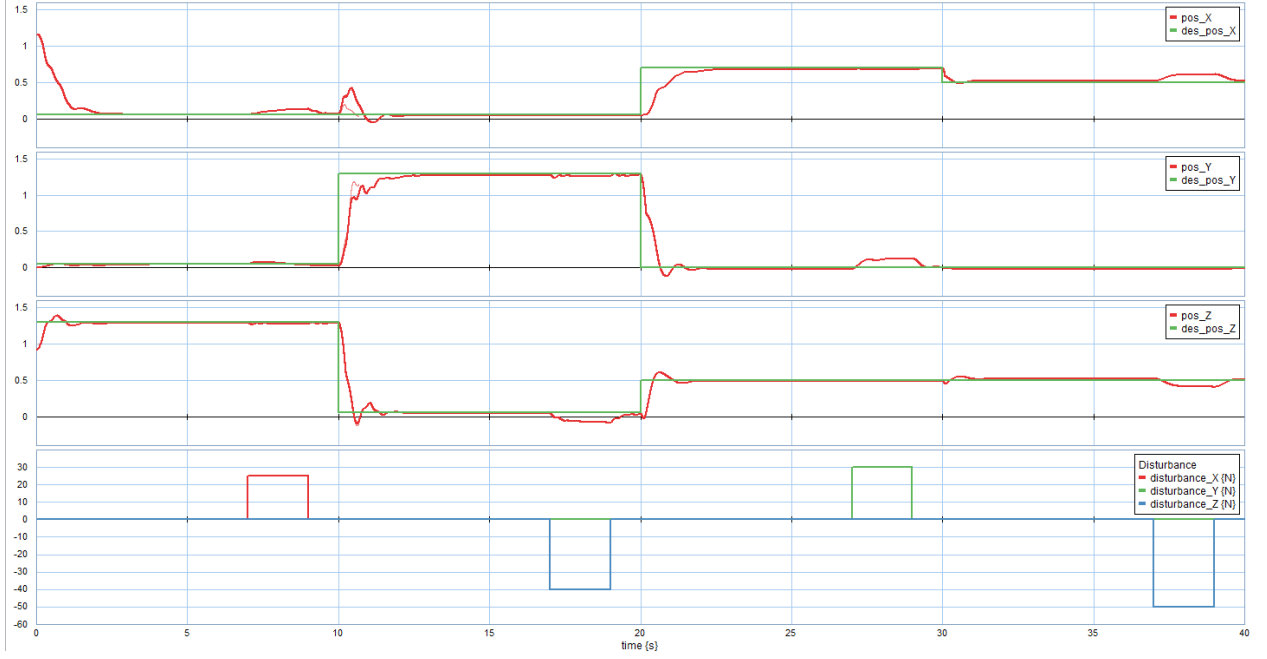


Figure 27: Plot with the impedance test for the discrete domain
G06_disc_impedance_test.emx

4.1.1 Trajectory tracking and integration error

During the simulation of the end-effector's dynamic trajectory, a steady-state tracking error was observed along the z-axis, preventing the arm from fully reaching the dynamic reference height. Interestingly, the system successfully tracks and holds a fixed static reference point in space without issue. This discrepancy between dynamic and static tracking performance can be partially attributed to the numerical solver; as the simulation progresses, the maximum integration error within the solver bounds accumulates, directly influencing the magnitude of this positional offset. However, from a control perspective, dynamic tracking errors are also a common symptom of phase lag within the control loop. While the current PI tuning successfully achieves static positioning, future iterations could investigate whether increasing the integral gain (k_i) or implementing a velocity feedforward term might mitigate the tracking delay.

4.1.2 Actuator tuning and friction estimation

Significant adjustments were required during the estimation of the gearbox friction and the tuning of the control loops. Initially, the gearbox resistance was calculated using the maximum nominal output torque rather than the input torque. Because this value was taken post-transmission, the derived friction was approximately two orders of magnitude too high. Recalculating this using the proper input torque resolved the severe damping issues.

Furthermore, the initial tuning of the PI controllers was overly aggressive. We found that calculating the expected required current (in Amperes) for a given reference torque input provided a much more realistic baseline for the order of magnitude required for our gains. Finally, we experienced early instability due to improperly capped control gains. By implementing strict upper limits on the effort sources, especially capping the appropriate maximum voltage to 48 V and continuous power ensured the simulated controllers used realistic hardware constraints.

4.1.3 Power limit motors exceeded

During the simulation and motor tuning phase, we noticed that motors 1 and 2 had to use significantly more power during trajectory tracking than their nominal values. The cause for this turned out

to be due to a mistake in the motor/gearbox friction. However, this is exactly the purpose of the simulation/verification: to see if the chosen components can handle the required load cases. In the case that the power limit was indeed exceeded, we could have modelled the thermal domain and added something like liquid cooling, or even come up with a solution like two motors in parallel. With the thermal model, we could have found out if exceeding the limit for a short time is fine, and then if throttling would be enough if the temperature would become too high.

In our validation simulations, we observed that when the outer impedance control loop is disabled, the robotic arm fails to perfectly hold its position when actuated by series elastic actuators, contrasting with the successful stationary hold of perfect actuators. We identified three primary contributing factors for this positional drift:

- **Actuator reaction time and spring energy storage:** Realistic actuators show a non-zero reaction time, causing a slight lag in gravity compensation at any given instant. As a result, the arm drops marginally before the torque controller can output the required flow. Furthermore, a portion of the control effort needed to hold the arm up is instead absorbed and stored as elastic energy within the physical spring of the SEA, leaving the actual joint under-compensated. Because the gravity compensation is based on the current configuration, a continuous cycle of slight dropping and delayed compensation is created until internal frictional effects finally stabilize the motion.
- **Velocity omission in feedforward:** The current gravity compensation model is purely position-dependent and does not account for the joint velocities present while the arm is still settling. This hypothesis is supported by the observation that if the arm is brought to a complete static halt before the impedance control is disabled, the robotic arm remains standing upright successfully without drifting.
- **Simulation and integration drift:** Finally, minor solver inaccuracies add to the drift over time. This was verified by increasing the overall simulation accuracy settings, which yielded a corresponding reduction in the magnitude of the arm's fall.

4.1.4 Feedforward friction compensation

While the static and viscous friction of the motor and gearbox were accurately modelled utilizing an 80/20 Coulomb-viscous ratio, the current torque control architecture relies purely on PI feedback error to overcome these resistive forces. This inherent feedback delay contributes to the observed phase lag during dynamic trajectory tracking. To solve this, a friction compensation feedforward term could be added directly into the current controller. By utilizing the previously derived friction parameters, the required feedforward current i_{ff} can be pre-calculated based on the desired angular velocity $\dot{q}_d$ and the motor's torque constant K_t :

$$i_{ff} = \frac{1}{K_t} (\tau_c \tanh(\gamma \dot{q}_d) + b \dot{q}_d) \quad (14)$$

Implementing this feedforward term would supply the necessary effort to overcome static breakaway friction and dynamic viscous drag, significantly unburdening the integral term of the PI controller and reducing steady-state dynamic tracking error. Such an implementation could be considered when expanding control system.

4.2 Conclusion

In this project, a 3-DOF robotic arm was successfully modelled and simulated using series elastic actuators for cartesian impedance control. By incrementally verifying the model from a basic unactuated system to a discrete-time, realistically actuated system, we gained practical insights

into the complexities of physical modelling and control in 20-sim. The main takeaways from this modelling and simulation process are:

- SEA effectiveness: The implemented SEAs successfully mimic a virtual spring-damper system. They reliably absorb external disturbances and cap the interaction forces to safe levels suitable for human interactions.
- Actuator and friction modelling: Accurately modelling the mechanical transmission proved essential. Correctly defining the gearbox input versus output torque resolved initial friction estimation issues that caused severe over-damping. Furthermore, applying physical power and voltage limits to the effort sources was necessary to ensure realistic controller behaviour.
- Control and integration limitations: Both static reference tracking and dynamic trajectory tracking performed accurately. However, minor simulation drift was observed during unactuated states, highlighting the system's sensitivity to accumulating integration errors from the numerical solver over time.
- Discretisation: Transitioning to a discrete-time model highlighted the necessity of domain-specific sampling rates (10 kHz for the electrical loop, 1 kHz for the mechanical loops) and required PI re-tuning for the joints to maintain overall stability.

Ultimately, despite minor simulation drift, the model fulfils its primary goal: providing a robust, realistic, and rigorously verified simulation of an impedance-controlled robotic arm.

References

- [1] maxon Group, *Maxon online catalog: Compact drives, motors, gears, sensors*, <https://www.maxongroup.us/maxon/view/catalog/>, Accessed: 2026-06-16, 2026.
- [2] AMD Machines. "Iso 10218 & iso/ts 15066 explained: Robot safety standards for integrators." Published January 10, 2026, AMD Machines, Accessed: Jun. 16, 2026. [Online]. Available: <https://amdmachines.com/blog/robot-safety-standards-iso-10218-and-ts-15066-explained/>.
- [3] Wikipedia contributors, *List of moments of inertia — wikipedia, the free encyclopedia*, https://en.wikipedia.org/wiki/List_of_moments_of_inertia, Accessed: 2025-06-16.